

Feedback

Workshop 9

Workshop 9 feedback

- The ninth workshop looked at dense matrix-matrix multiplication
- We compared DGEMM with a simple implementation using nested for loops
- The DGEMM implementation comes from Intel's Math Kernel Library and has been optimised for the processors used in Isca's compute nodes

Workshop 9

matmul.c **$C := C + AB$**

```
int main(){  
    const int N=3;      // Dimensions of matrices are NxN  
    int i, j, k;         // Loop indices  
    double *A, *B, *C;   // Matrices A, B and C  
  
    /* Allocate memory to store the matrices */  
    A = (double *) malloc(N*N*sizeof(double));  
    B = (double *) malloc(N*N*sizeof(double));  
    C = (double *) malloc(N*N*sizeof(double));  
  
    /* Initialise matrices A, B and C */  
    for (i=0; i<N; i++){  
        for (j=0; j<N; j++){  
            A[j+N*i] = (double) j+N*i;  
            B[j+N*i] = (double) j+1;  
            C[j+N*i] = 0.0;  
        }  
    }  
  
    /* Multiply matrices using nested for loops: C=AB */  
    for (i=0; i<N; i++){  
        for (j=0; j<N; j++){  
            for (k=0; k<N; k++){  
                C[j+N*i] = C[j+N*i] + A[k+N*i] * B[j+N*k];  
            }  
        }  
    }  
  
    /* Print out arrays to check the results */  
    printf("Matrix A:\n");  
    for (i=0; i<N; i++){  
        for (j=0; j<N; j++){  
            printf("%3g ", A[j+N*i]);  
        }  
        printf("\n");  
    }  
}
```

> ./matmul

Matrix A:

0 1 2
3 4 5
6 7 8

Matrix B:

1 2 3
1 2 3
1 2 3

Matrix C:

3 6 9
12 24 36
21 42 63

>

Workshop 9

dgemm.c

$$C \leftarrow \alpha AB + \beta C$$

```
int main(){
    const int N=3;      // Dimensions of matrices are NxN
    int i, j;           // Loop indices
    double *A, *B, *C;  // Matrices A, B and C

    /* alpha and beta coefficients in DGEMM calculation */
    const double alpha=1.0;
    const double beta=0.0;

    /* Allocate memory to store the matrices */
    A = (double *) malloc(N*N*sizeof(double));
    B = (double *) malloc(N*N*sizeof(double));
    C = (double *) malloc(N*N*sizeof(double));

    /* Initialise matrices A, B and C */
    for (i=0; i<N; i++){
        for (j=0; j<N; j++){
            A[j+N*i] = (double) j+N*i;
            B[j+N*i] = (double) j+1;
            C[j+N*i] = 0.0;
        }
    }

    /* Call DGEMM to multiply the arrays: C=AB */
    cblas_dgemm(CblasRowMajor, CblasNoTrans, CblasNoTrans,
                N, N, N, alpha, A, N, B, N, beta, C, N);

    /* Print out arrays to check the results */
    printf("Matrix A:\n");
    for (i=0; i<N; i++){
        for(j=0; j<N; j++){
            printf("%3g ", A[j+N*i]);
        }
    }
}
```

```
> ./dgemm
Matrix A:
  0  1  2
  3  4  5
  6  7  8
Matrix B:
  1  2  3
  1  2  3
  1  2  3
Matrix C:
  3  6  9
 12 24 36
 21 42 63
>
```

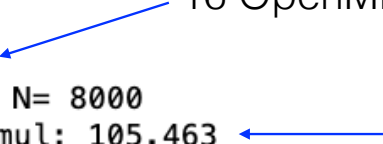
Workshop 9

Question 1: How long does the `matmul` program take to multiply two 8000×8000 matrices?

Threads: 16
Matrix size: N= 8000
Time for matmul: 105.463
Flops: 9.7096 GFlops
C[100]: 3.2316e+09

16 OpenMP threads

105.5 seconds



Workshop 9

Question 2: How long does the DGEMM program take to multiply two 8000×8000 matrices and how much faster it is than the `matmul` program?

```
Threads: 16  
Matrix size: N= 8000  
Time for DGEMM: 2.52175 ← 2.5 seconds  
Flops: 406.066 GFlops  
C[100]: 3.2316e+09
```

$$\frac{105.5}{2.5} = 42.4 \rightarrow \text{DGEMM is 42 times faster}$$

Workshop 9

Question: 3 What fraction of peak performance does the DGEMM program achieve? The processor can deliver 16 operations per cycle and the base clock frequency is 2.60 GHz.

Running with threads= 2
Matrix size: N= 8000
Time for DGEMM: 11.108
Flops: 92.1858 GFlops ← 2 cores
C[100]: 3.2316e+09

$$R_{\text{peak}} = N_{\text{cores}} R_{\text{clock}} N_{\text{operations/cycle}}$$

Running with threads= 4
Matrix size: N= 8000
Time for DGEMM: 5.82602
Flops: 175.763 GFlops ← 4 cores
C[100]: 3.2316e+09

Running with threads= 8
Matrix size: N= 8000
Time for DGEMM: 3.98055
Flops: 257.251 GFlops ← 8 cores
C[100]: 3.2316e+09

Running with threads= 12
Matrix size: N= 8000
Time for DGEMM: 2.5628
Flops: 399.564 GFlops ← 12 cores
C[100]: 3.2316e+09

Running with threads= 16
Matrix size: N= 8000
Time for DGEMM: 2.65898
Flops: 385.11 GFlops ← 16 cores
C[100]: 3.2316e+09

Workshop 9

Max. turbo frequency = 3.40GHz

No. of Cores	DGEMM performance (Gflops)	R_{peak} (Gflops)	Fraction of R_{peak}
2	92.1858	83.2	1.108
4	175.763	166.4	1.056
8	257.251	332.8	0.773
12	399.564	499.2	0.800
16	385.11	665.6	0.579

Workshop 9

Question: 4 How does the measured performance (flops) depend on the size of the matrix and is this what you would expect?

Threads: 8
Running with size 4000
Matrix size: N= 4000
Time for DGEMM: 0.871059
Flops: 146.948 GFlops
C[100]: 8.07798e+08

4000x4000

$R_{\text{peak}} = 332.8 \text{ Gflops}$

Running with size 6000
Matrix size: N= 6000
Time for DGEMM: 1.65285
Flops: 261.366 GFlops
C[100]: 1.8177e+09

6000x6000

Running with size 8000
Matrix size: N= 8000
Time for DGEMM: 3.32247
Flops: 308.205 GFlops
C[100]: 3.2316e+09

8000x8000

Increasing flops as
the matrix size
increases

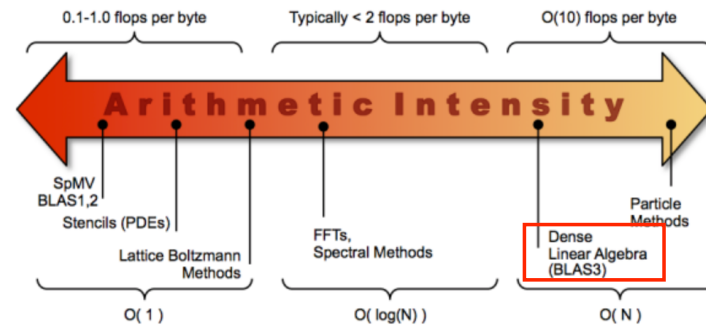
Running with size 10000
Matrix size: N= 10000
Time for DGEMM: 6.20929
Flops: 322.098 GFlops
C[100]: 5.0495e+09

10000x10000

Running with size 12000
Matrix size: N= 12000
Time for DGEMM: 11.2535
Flops: 307.104 GFlops
C[100]: 7.27139e+09

12000x12000

Workshop 9



Arithmetic intensity increases with problem size
so we expect more flops as the matrices get larger

Workshop 9 summary

- Calling an optimised **DGEMM** to multiply matrices is much faster than a simple implementation using nested for loops
- Multiplying large dense matrices can achieve a high fraction of peak performance
- For more on the optimisations used see Pawel Gepner, Victor Gamayunov, David L. Fraser, “Effective Implementation of DGEMM on Modern Multicore CPU”, Procedia Computer Science, Volume 9, 2012, Pages 126-135, <https://doi.org/10.1016/j.procs.2012.04.014>